Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

Bonnes pratiques

Organiser son code pour qu'il reste facile à changer

Speaker notes

Fil rouge : FraisDirect, l'épicerie en ligne des autres cours. Son code a grossi, et chaque modification devient plus risquée. On voit trois façons de garder le code facile à changer : découper les objets, designer l'API, ranger les packages.

Le code de FraisDirect a grossi

- Une classe `Client` de 40 champs
- Un `CommandeService` qui envoie aussi les mails et les factures
- Une API qui renvoie tout, tout le temps
- Un package `service/` de 80 classes

Rien n'est faux. Mais chaque modification prend une journée.

Speaker notes

Demander à la salle s'ils ont déjà vu ça en entreprise ou en projet. Chaque point correspond à une partie du cours.

Au programme

1. Découper ses objets : **SOLID** et la **composition**
2. Designer son **API REST**
3. Ranger ses **packages**

Speaker notes

On commence par un petit exercice : il servira d'exemple pour la composition.

ACTE 1

Découper ses objets

Des classes petites, qui font une seule chose

Speaker notes

On part d'un exercice qui a l'air simple et qui devient vite pénible si on l'aborde avec l'héritage.

EXERCICE

Petit exercice

1. Implémenter un objet `Afficheur` avec une méthode `afficher(String message)` qui fait `System.out.println(message)`
2. Ajouter une vérification : si le message contient un « a », on ne l'affiche pas
3. Dans un `main` : deux afficheurs, un en minuscules et un en majuscules
4. L'afficheur en majuscules vérifie aussi que le message fait 5 caractères
5. Un troisième affiche normalement et vérifie **uniquement** la taille de 5

Speaker notes

Dérouler une consigne à la fois, et laisser les étudiants coder après chacune. La dernière consigne est le piège : elle ne veut pas la vérification du « a ». Ceux qui ont fait de l'héritage doivent tout défaire. On garde leurs solutions pour la partie composition.

Les principes SOLID

S	Single responsibility	Une classe, une seule raison de changer
O	Open/closed	Ajouter un comportement sans modifier l'existant
L	Liskov substitution	Une sous-classe peut remplacer sa mère
I	Interface segregation	Des petites interfaces plutôt qu'une grosse
D	Dependency inversion	Dépendre d'interfaces, pas d'implémentations

Speaker notes

Cinq principes, une phrase chacun. On ne détaille que le S, celui qu'on viole le plus souvent. Le D, ils l'ont déjà utilisé sans le savoir dans le cours sur les tests : c'est lui qui permet de passer un mock à CommandeService.

QUESTION

Une classe `ClientService` gère les clients, envoie les mails et génère les factures PDF. Quel principe viole-t-elle surtout ?

A. S : Single responsibility

B. O : Open/closed

C. L : Liskov substitution

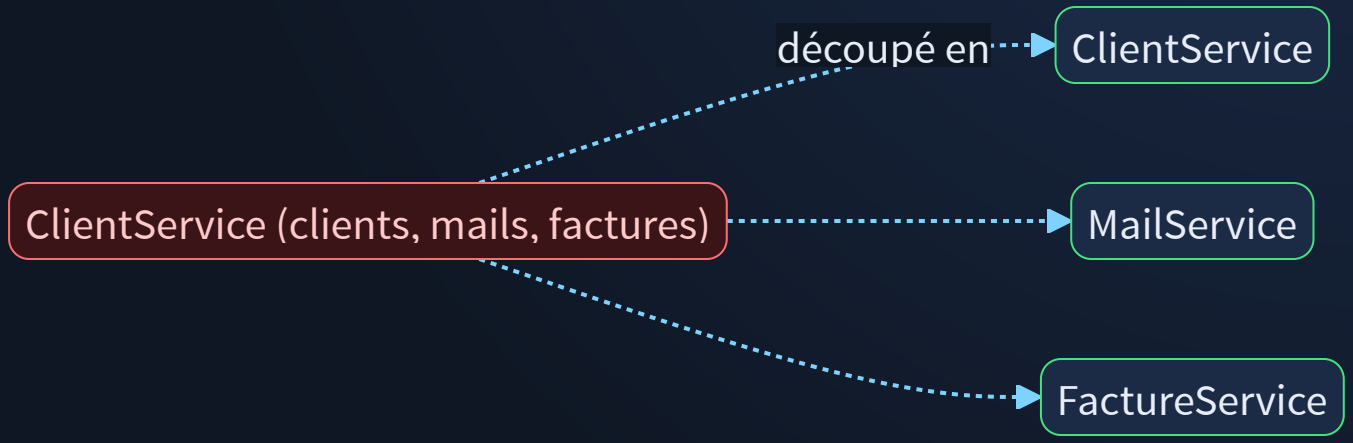
D. I : Interface segregation

E. D : Dependency inversion

Speaker notes

Réponse A. Trois raisons de changer : les règles client, le contenu des mails, le format des factures. Un changement de mise en page de la facture oblige à toucher la classe qui gère les clients.

S : découper un service



- Chaque classe a une seule raison de changer
- Chaque classe se teste seule, avec moins de mocks

Speaker notes

Lien avec le cours sur les tests : une classe qui fait trois choses a trois fois plus de dépendances à mocker. Une classe dure à tester est souvent une classe trop grosse.

S : les entités aussi

Les entités sont souvent la racine du problème

```
public class Client {
    private String nom;
    private String rue;
    private String ville;
    private String codePostal;
}
```

La rue, la ville et le code postal vont toujours ensemble…

Speaker notes

Demander ce qu'ils remarquent. Trois champs qui n'ont de sens qu'ensemble : c'est un objet qui n'a pas encore de nom.

Étape 1 · Un groupe logique

```java
public class Client {
    private String nom;
    private Adresse adresse;
}

public class Adresse {
    private String rue;
    private String ville;
    private String codePostal;
}
```

Speaker notes

L'adresse devient un objet. La validation du code postal, par exemple, a maintenant une place évidente : dans Adresse.

Étape 2 · Le besoin change

FraisDirect livre à une adresse et facture à une autre

```
public class Client {
    private String nom;
    private Adresse adresseLivraison;
    private Adresse adresseFacturation;
}
```

Une ligne de plus, au lieu de quatre champs copiés.

Speaker notes

Sans l'étape 1, il aurait fallu rueFacturation, villeFacturation, codePostalFacturation… et dupliquer la validation. Le découpage paie dès le premier changement.

Étape 3 · Des rôles différents

Un client et un livreur ont tous les deux un compte

```java
public class Utilisateur {
    private String nom;
    private String email;
    private String motDePasse;
}

public class Client {
    private Utilisateur utilisateur;
    private Adresse adresseLivraison;
}

public class Livreur {
    private Utilisateur utilisateur;
    private String immatriculation;
}
```

Speaker notes

Client et Livreur contiennent un Utilisateur, ils n'en héritent pas. C'est de la composition, le sujet de la suite.

Composition over inheritance

- L'héritage n'est pas la solution pour factoriser du code
- 90 % du temps, il est mal utilisé (source : moi)

Speaker notes

Le réflexe classique : deux classes ont du code en commun, donc on crée une classe abstraite. On va voir pourquoi c'est rarement une bonne idée, et le cas où ça l'est.

Mauvais exemple

```java
abstract class Notification {
    abstract void envoyer(Commande commande);

    protected String formaterMontant(long centimes) {
        return String.format("%d,%02d €",
                centimes / 100, centimes % 100);
    }
}

class NotificationMail extends Notification {
    void envoyer(Commande commande) {
        String montant = formaterMontant(commande.montant());
        // … envoie le mail
    }
}
```

La mère n'appelle jamais `envoyer` : elle sert juste de boîte à outils

Speaker notes

Conséquence : FactureService veut aussi formater un montant. Il doit hériter de Notification ? Non. Et en Java, on n'hérite que d'une seule classe. Le code partagé doit aller dans sa propre classe.

Bon exemple

```
abstract class Notification {
    final void notifier(Commande commande) {
        if (accepte(commande.client())) {
            envoyer(commande);
        }
    }

    abstract boolean accepte(Client client);
    abstract void envoyer(Commande commande);
}

class NotificationSms extends Notification {
    boolean accepte(Client client) {
        return client.accepteLesSms();
    }

    void envoyer(Commande commande) {
        // … envoie le SMS
    }
}
```

Speaker notes

Ici la mère appelle les méthodes abstraites : elle définit le déroulé, les filles remplissent les trous. C'est le bon usage d'une classe abstraite (le patron « template method »).

Retour à l'afficheur

Le format et la vérification deviennent des objets

```
class Afficheur {
    private final Function<String, String> format;
    private final Predicate<String> verification;

    Afficheur(Function<String, String> format,
              Predicate<String> verification) {
        this.format = format;
        this.verification = verification;
    }

    void afficher(String message) {
        if (verification.test(message)) {
            System.out.println(format.apply(message));
        }
    }
}
```

Speaker notes

Correction de l'exercice. Plus aucune sous-classe : un afficheur est assemblé à partir d'un format et d'une vérification.

Retour à l'afficheur

```
1  Predicate<String> sansA = m -> !m.contains("a");
2  Predicate<String> cinqCaracteres = m -> m.length() == 5;
3
4  var minuscule = new Afficheur(String::toLowerCase, sansA);
5
6  var majuscule = new Afficheur(
7          String::toUpperCase,
8          sansA.and(cinqCaracteres));
9
10 var normal = new Afficheur(
11         Function.identity(), cinqCaracteres);
```

La consigne piège devient une ligne : on assemble autrement

Speaker notes

Avec l'héritage, le troisième afficheur cassait toute la hiérarchie car il ne voulait pas la vérification du « a ». Avec la composition, on choisit les briques. Comparer avec les solutions des étudiants.

À retenir

- Une classe, une seule raison de changer
- Des champs qui vont ensemble forment un objet
- Pour partager du code : une classe à part, injectée
- Une classe abstraite seulement si la mère appelle les méthodes des filles

Speaker notes

Le test simple : si la classe mère n'appelle jamais ses méthodes abstraites, l'héritage ne sert qu'à partager du code, et la composition fait mieux.

ACTE 2

Designer son API REST

Ce que voit le front n'est pas ce qui est en base

Speaker notes

L'API est un contrat avec le front et les autres équipes. Une fois publiée, elle est difficile à changer : autant la penser dès le début.

QUESTION

Retourner directement l'entité JPA `Client` (avec son mot de passe) depuis un `@RestController`, bonne idée ?

Speaker notes

Réponse : non. Faire justifier les réponses avant de passer à la suite.

Ce qui se passe

```java
@Entity
public class Client {
    @Id
    private Long id;
    private String email;
    private String motDePasse;
    @OneToMany(mappedBy = "client")
    private List<Commande> commandes;
}
```

- Le hash du mot de passe part dans le JSON
- Client → commandes → client → commandes… : sérialisation sans fin
- Renommer une colonne casse le front

Speaker notes

Les trois problèmes arrivent pour de vrai. Le deuxième se termine en StackOverflowError ou en LazyInitializationException. Le troisième est le plus sournois : le schéma de la base devient le contrat de l'API.

La solution : un DTO

```java
public record ClientDto(Long id, String email) {}

@GetMapping("/clients/{id}")
public ClientDto trouver(@PathVariable Long id) {
    Client client = clientService.trouver(id);
    return new ClientDto(client.getId(), client.getEmail());
}
```

- On choisit exactement ce qui sort
- On maîtrise le schéma, et on peut le versionner

Speaker notes

DTO : Data Transfer Object. Un record suffit. La conversion peut aussi se faire avec MapStruct, mais à la main c'est très bien pour commencer.

Découper les ressources

Un produit de FraisDirect

```
1 public class Produit {
2     private Long id;
3     private String nom;
4     private Categorie categorie;
5     private List<Produit> similaires;
6     private List<Avis> avis;
7 }
```

Speaker notes

Un petit objet (la catégorie), une liste d'objets du même type (les produits similaires), une liste qui peut devenir très longue (les avis). Que renvoyer sur GET /produits/12 ?

Mauvaise pratique

`GET /produits/12`

```
{
  "id": 12,
  "nom": "Comté 18 mois",
  "categorie": {
    "nom": "Fromages"
  },
  "similaires": [
    { "id": 13, "nom": "Comté 24 mois" }
  ],
  "avis": [
    { "note": 5, "texte": "Délicieux" },
    …
  ]
}
```

Speaker notes

Dérouler avec les fragments. La catégorie, ça va. Les produits similaires et les avis : le front qui veut juste afficher le nom télécharge 3 000 avis.

Bonne pratique

Speaker notes

Une ressource par URL. Le petit objet reste dans le produit, les listes ont leur propre URL, avec filtre et pagination. Le front appelle ce dont il a besoin.

Les verbes HTTP

Je veux	Appel
Lire	`GET /commandes/12`
Chercher	`GET /commandes?client=42`
Créer	`POST /commandes`
Modifier	`PUT /commandes/12`
Supprimer	`DELETE /commandes/12`

Speaker notes

L'URL désigne la ressource, le verbe dit ce qu'on en fait.

Les verbes dans l'URL

- Pour une action métier : `POST /commandes/12/annuler`
- `/init` : le `POST` suffit
- `/search` : le `GET` suffit
- `/delete` : le verbe HTTP existe

Speaker notes

Annuler une commande, ce n'est pas la supprimer : on la garde, on change son état, on rembourse. C'est une action métier, un verbe dans l'URL est justifié. Pour le reste, les verbes HTTP suffisent.

QUESTION

Quel appel pour supprimer le produit 12 ?

A. `POST /produits/12/delete`

B. `DELETE /produits/12`

C. `GET /produits/delete?id=12`

D. `POST /produits/delete/12`

Speaker notes

Réponse B. La C est la pire : un GET ne doit jamais modifier quoi que ce soit, un robot d'indexation ou un préchargement du navigateur pourrait supprimer le produit.

À retenir

- Jamais d'entité dans un contrôleur : un DTO
- Une ressource par URL, les listes à part
- Le verbe HTTP dit l'action, sauf pour une action métier

Speaker notes

Le DTO est aussi une question de sécurité : c'est lui qui garantit que le mot de passe ne sort pas.

ACTE 3

Ranger ses packages

Trouver le code sans le chercher

Speaker notes

Trois façons de ranger le même projet, de la plus courante à la plus utile.

Rangement 1 · Par couche

```
fraisdirect/
├─ config/
├─ controller/
├─ repository/
└─ service/
```

- ⊕ Simple pour démarrer
- ⊖ 80 classes dans `service/`
- ⊖ Toucher aux commandes = ouvrir 4 dossiers

Speaker notes

Le rangement de tous les tutoriels Spring. Très bien pour 10 classes, pénible à 200.

Rangement 2 · Par domaine

```
fraisdirect/
├─ catalogue/
│  ├─ controller/
│  ├─ repository/
│  └─ service/
├─ commande/
│  ├─ controller/
│  ├─ repository/
│  └─ service/
└─ livraison/
   └─ …
```

- Les éléments sont regroupés par domaine fonctionnel
- On explore le projet par le métier

Speaker notes

Un nouveau développeur qui doit corriger un bug sur les commandes sait où aller. Et un domaine qui grossit peut devenir une application séparée plus facilement.

Rangement 3 · Domaine, entrées et sorties

```
commande/
├─ config/
├─ in/
│  ├─ controller/
│  └─ amqp/
├─ out/
│  ├─ repository/
│  └─ paiement/
└─ service/
```

- `in/` : ce qui appelle le domaine (HTTP, messages)
- `out/` : ce que le domaine appelle (base, API externes)
- ⊕ Les dépendances se voient
- ⊕ Les études d'impact sont plus faciles

Speaker notes

Question d'étude d'impact : la banque change son API, qu'est-ce qui est touché ? Uniquement out/paiement. Le listener RabbitMQ du cours middleware irait dans in/amqp.

À retenir

- Ranger par domaine, pas par couche
- Séparer ce qui entre et ce qui sort du domaine
- Le but : trouver le code et mesurer l'impact d'un changement

Speaker notes

Fin du cours. Les trois parties disent la même chose : du code découpé selon le métier est du code qu'on peut changer sans peur.